

蓝桥杯30天算法冲刺集训



Day-15 图与最短路

图的种类

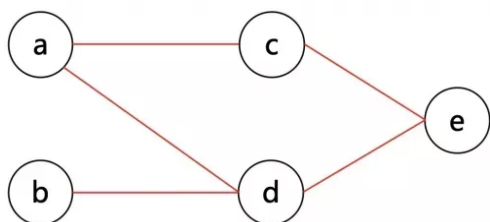
图有多种，包括 **无向图 (Undirected graph)**，**有向图 (Directed graph)** 等

若 G 为无向图，则 E 中的每个元素为一个无序二元组 (u, v) ，称作 **无向边 (Undirected edge)**，简称 **边 (Edge)**，其中 $u, v \in V$ 。设 $e = (u, v)$ ，则 u 和 v 称为 e 的 **端点 (Endpoint)**。

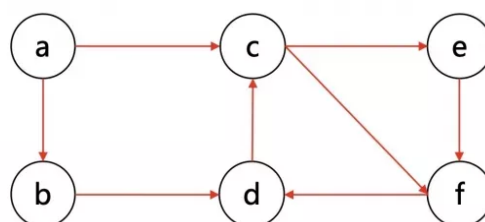
若 G 为混合图，则 E 中既有向边，又有无向边。

若 G 的每条边 $e_k = (u_k, v_k)$ 都被赋予一个数作为该边的 **权**，则称 G 为 **赋权图**。如果这些权都是正实数，就称 G 为 **正权图**。

形象地说，图是由若干点以及连接点与点的边构成的。



无向图



有向图

度数

与一个顶点 v 关联的边的条数称作该顶点的 **度 (Degree)**，记作 $d(v)$ 。特别地，对于边 (v, v) ，则每条这样的边要对 $d(v)$ 产生 2 的贡献。

对于无向简单图，有 $d(v) = |N(v)|$ 。

握手定理 (又称图论基本定理)：对于任何无向图 $G = (V, E)$ ，有 $\sum_{v \in V} d(v) = 2|E|$ ，即无向图中结点度数的总和等于边数的两倍。有向图中结点的入度之和等于出度之和等于边数。

推论：在任意图中，度数为奇数的点必然有偶数个。

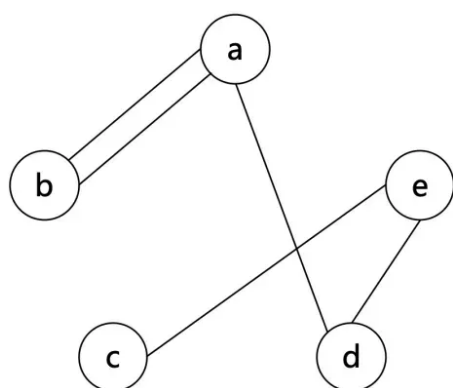
简单图

自环 (Loop)：对 E 中的边 $e = (u, v)$ ，若 $u = v$ ，则 e 被称作一个自环。

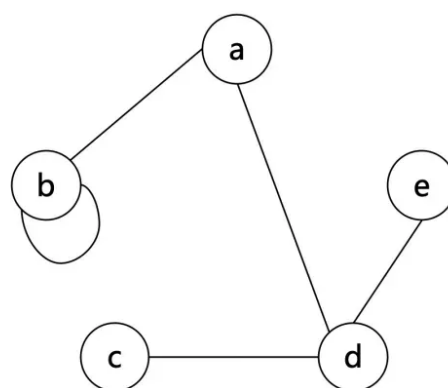
重边/平行边 (Multiple edge)：若 E 中存在两个完全相同的元素（边） e_1, e_2 ，则它们被称作（一组）重边。

简单图 (Simple graph)：若一个图中 **没有自环和重边**，它被称为简单图。
非空简单无向图中一定存在度相同的结点。

如果一张图中有自环或重边，则称它为 **多重图 (Multigraph)**。



平行边



自环边

在无向图中 (u, v) 和 (v, u) 算一组重边，而在有向图中， $u \rightarrow v$ 和 $v \rightarrow u$ 不为重边。

在题目中，如果没有特殊说明，是可以存在自环和重边的，在做题时需特殊考虑。

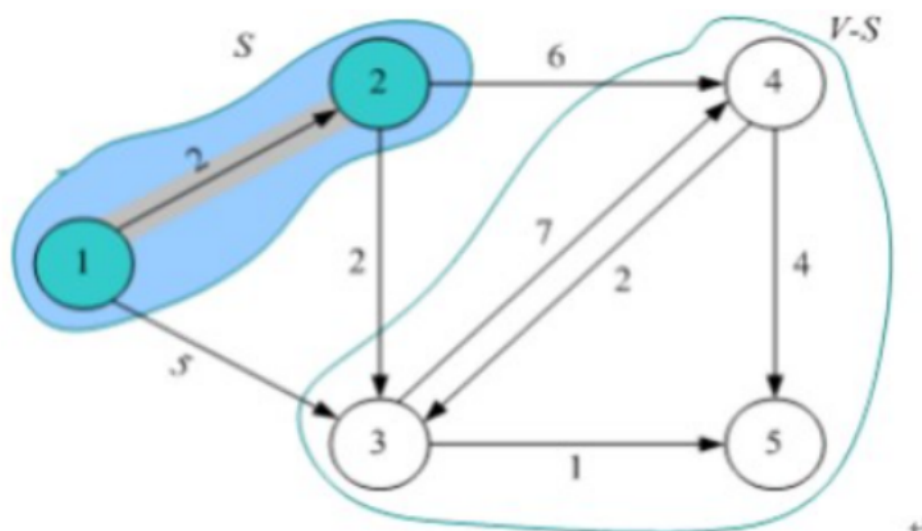
朴素Dijkstra算法

算法思想

Dijkstra 算法是解决**单源最短路径问题**的贪心算法，**它先求出长度最短的一条路径，再参照该最短路径求出长度次短的一条路径，直到求出源点到其他各个节点的最短路径**

Dijkstra 算法基本思想：将顶点集合 V 划分为两部分：集合 S 和集合 $V - S$ ，其中 S 中的顶点到源点的最短路径已经确定， $V - S$ 中的节点到源点的最短路径待定。

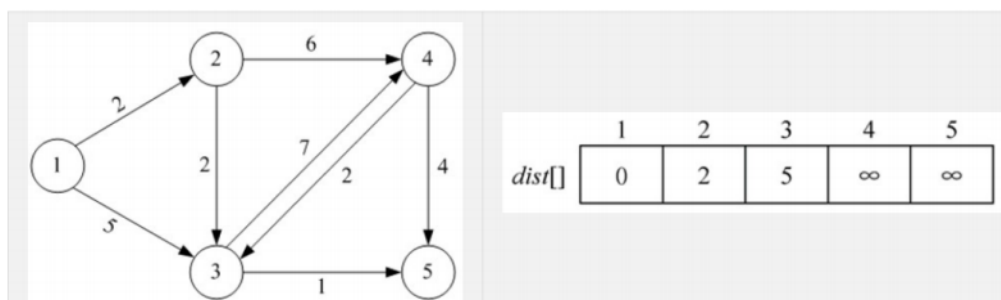
从源点出发只经过 S 中的节点到达 $V - S$ 中的顶点的路径称为特殊路径。*Dijkstra* 算法的贪心策略是选择最短的特殊路径长度 $dist[t]$ ，并将顶点 t 加入到集合 S 中，同时借助 t 更新数组 $dist[]$ 。一旦 S 包含了所有顶点， $dist[]$ 就是从源点到其他节点的最短路径长度



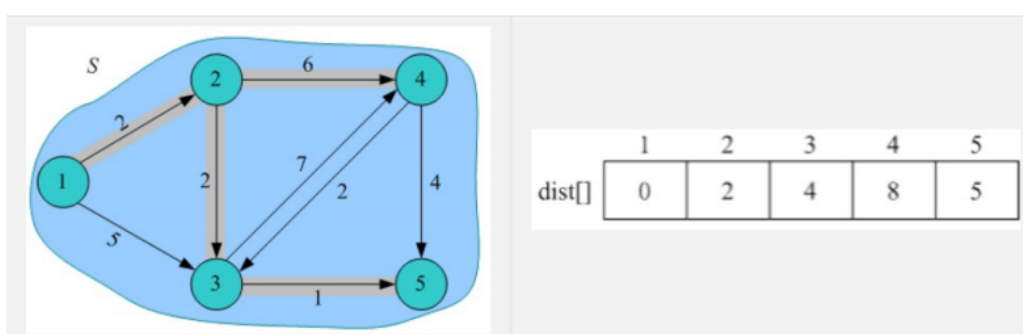
算法设计

1. **数据结构**：邻接矩阵 $G[][]$ 存储图， $dist[i]$ 记录从源点到节点 i 的最短路径长度，如果 $flag[i]$ 等于 $true$ ，说明节点 i 已加入集合 S ，否则 i 属于集合 $V - S$ 。
2. **初始化**：假设 u 为源点，令集合 $S = u$ ，对 $V - S$ 集合中的节点 i ，初始化 $dist[i] = G[u][i]$
3. **找最小**：按照贪心策略来查找 $V - S$ 集合中 $dist[]$ 最小的顶点 t ， t 就是 $V - S$ 集合中距离源点 u 最近的顶点。将顶点 t 加入集合 S 中
4. **松弛操作**：对 $V - S$ 集合中所有节点 j ，考察是否可以借助 t 得到更短的路径。如果源点 u 经过 t 到 j 的路径更短，即 $dist[j] > dist[t] + G[t][j]$ ，则更新 $dist[j] = dist[t] + G[t][j]$ ，即松弛操作
5. 重复执行步骤 3 和步骤 4，直到 $V - S$ 为空

[例题]求源点1到其它各个顶点的最短路径



过程同学们自己模拟上述算法设计步骤计算，最终结果为



堆优化版本Dijkstra算法

朴素版Dijkstra算法找最小值方法

```
int temp=INF,t;
for(int j=1;j<=n;j++)//在集合V-S中寻找距离源点u最近的顶点t
{
    if(flag[j]==0&&dist[j]<temp)
    {
        t=j;
        temp=dist[j];
    }
}
```

按照 **贪心策略** 查找 $V - S$ 集合中 $dist[]$ 最小的顶点 t ，其时间复杂度为 $O(n)$ ，如果使用 **优先队列**，则每次找最小值时间复杂度降为 $O(\log n)$ ，找最小值的总时间复杂度为 $O(n \log n)$

邻接矩阵写法

```

#include <bits/stdc++.h>
using namespace std;

const int N = 1005;
const int INF = 0x3f3f3f3f; // 无穷大
int g[N][N], dist[N]; // g[][]为邻接矩阵, dist[i]表示源点到结点i
的最短路径长度
int n, m; // n为顶点数, m为边数
bool flag[N]; // 如果flag[i]等于true, 说明结点i已经加入到S集合; 否
则i属于V-S集合

struct node {
    int v, dis; // 顶点v, 源点到v的最短路径长度dis
    bool operator < (const node &a) const { // 重载<, 优先队列优
        先级, dis越小越优先
        return dis > a.dis;
    }
};

void dijkstra(int u) {
    priority_queue<node> q; // 优先队列优化
    memset(dist, 0x3f, sizeof(dist));
    dist[u] = 0;
    q.push({u, 0});
    while (!q.empty()) {
        node it = q.top(); // 优先队列队头元素为dist最小值
        q.pop();
        int t = it.v;
        if (flag[t]) // 说明已经找到了最短距离, 该顶点是队列里面的
            重复元素
            continue;
        flag[t] = true;
        for (int j = 1; j <= n; j++) { // 松弛操作
            if (!flag[j] && dist[j] > dist[t] + g[t][j]) {
                dist[j] = dist[t] + g[t][j];
                q.push({j, dist[j]}); // 把更新后的最短距离压入优
                先队列, 注意: 里面的元素有重复
            }
        }
    }
}

```

```

    }
}

int main() {
    int u, v, w, st; // u,v表示顶点, w表示u--v的距离, st表示源点
    cin >> n >> m;
    memset(g, 0x3f, sizeof(g));
    while (m--) {
        cin >> u >> v >> w;
        g[u][v] = min(g[u][v], w); // 邻接矩阵储存, 保留最小的距离
    }
    // cin >> st; // 输入源点
    st = 1;
    dijkstra(st);
    if (dist[n] == INF)
        cout << -1;
    else
        cout << dist[n];
    return 0;
}

```

邻接表写法

```

#include <bits/stdc++.h>
using namespace std;

const int N = 1005; // 顶点数是多少具体看题目
const int INF = 0x3f3f3f3f; // 无穷大
int dist[N]; // dist[i]表示源点到结点i的最短路径长度
int n, m; // n为顶点数, m为边数
bool flag[N]; // 如果flag[i]等于true, 说明结点i已经加入到S集合; 否则i属于V-S集合

struct node {
    int v, w; // 到v的路径长度为w
    bool operator < (const node &a) const { // 重载<, 优先队列优先级, dis越小越优先
        return w > a.w;
    }
};

```

```

    }
};

vector<node> g[N];

void dijkstra(int u) {
    priority_queue<node> q; // 优先队列优化
    memset(dist, 0x3f, sizeof(dist));
    dist[u] = 0;
    q.push({u, 0});
    while (!q.empty()) {
        node it = q.top(); // 优先队列队头元素为dist最小值
        q.pop();
        int t = it.v;
        if (flag[t]) // 说明已经找到了最短距离，该结点是队列里面的
            重复元素
            continue;
        flag[t] = true;
        for (auto e : g[t]) // 松弛操作
        {
            int v = e.v, w = e.w;
            if (dist[v] > dist[t] + w) {
                dist[v] = dist[t] + w;
                q.push({v, dist[v]}); // 把更新后的最短距离压入优
            先队列，注意：里面的元素有重复
            }
        }
    }
}

int main() {
    int u, v, w, st; // u,v表示顶点,w表示u--v的距离,st表示源点
    cin >> n >> m;
    while (m--) {
        cin >> u >> v >> w;
        g[u].push_back({v, w});
    }
    // cin >> st; // 输入源点
    st = 1;

```



```
dijkstra(st);  
if (dist[n] == INF)  
    cout << -1;  
else  
    cout << dist[n];  
return 0;  
}
```

Floyd 算法

基本要点

1. 时间复杂度是 $O(n^3)$ ，因此，只适用于 $n < 500$ 的这种结点数不多的情况。
2. 它是多源最短路的，可以求出任何两点之间的最短路。
3. 支持负边权。
4. 能够判断图中是否存在负回路。

算法步骤

定义 $dis[i][j][k]$ 表示从 i 到 j 的可以经过 $1 \sim k$ 号结点的最短路径，此问题可以被分解为两个子问题

(1) 不经过 k 号结点，即 $dis[i][j][k-1]$

(2) 经过 k 号结点，即 $dis[i][k][k-1] + dis[k][j][k-1]$;

因此，有：

$$dis[i][j][k] = \min(dis[i][j][k-1], dis[i][k][k-1] + dis[k][j][k-1]);$$

上式中第三维 k 可以省略，则有：

$$dis[i][j] = \min(dis[i][j], dis[i][k] + dis[k][j]);$$

虽然 i, j, k 的顺序不保证，导致上式中 $dis[i][k]$ 和 $dis[k][j]$ 可能是已经被更新过的 $dis[i][k][k]$ 和 $dis[k][j][k]$ 了，

但是这并不影响其正确性。因为更新后的值一定会变得更小，不会影响最终结果。

Floyd 算法为什么把k放在最外层?

采用动态规划思想, $f[k][i][j]$ 表示 i 和 j 之间可以通过编号不超过 k , 即编号 $1, 2, \dots, k$ 的节点的最短路径。初值 $f[0][i][j]$ 为原图的邻接矩阵。

则该问题可以划分为两个子问题:

1. $f[k][i][j]$ 可以从 $f[k-1][i][j]$ 转移来, 表示 i 到 j 不经过 k 这个节点。
2. 也可以从 $f[k-1][i][k] + f[k-1][k][j]$ 转移过来, 表示经过 k 这个点。

于是: $f[k][i][j] = \min(f[k-1][i][j], f[k-1][i][k] + f[k-1][k][j])$

然后你就会发现 f 最外层一维空间可以省略, 因为 $f[k][\quad\quad]$ 只与 $f[k-1][\quad\quad]$ 有关, 虽然省略后, 无法确定 k, i, j 的关系, 但是 $f[k][i][k]$ 和 $f[k][k][j]$ 是由 $f[k-1][i][k]$ 和 $f[k-1][k][j]$ 计算而来, 因此前者一定小于等于后者, 省略后再取最小值并不会影响最终结果的正确性。

奶牛跨栏

```
#include <bits/stdc++.h>
using namespace std;

const int N = 3e2+5;
int n, m, t;
int g[N][N];

int main(){
    memset(g, 0x3f, sizeof g);
    cin >> n >> m >> t;
    for(int i = 1; i <= m; i++) {
        int s, e, h;
        cin >> s >> e >> h;
        g[s][e] = h;
    }
    // floyd
    for(int k = 1; k <= n; k++)
        for(int i = 1; i <= n; i++)
            for(int j = 1; j <= n; j++)
```

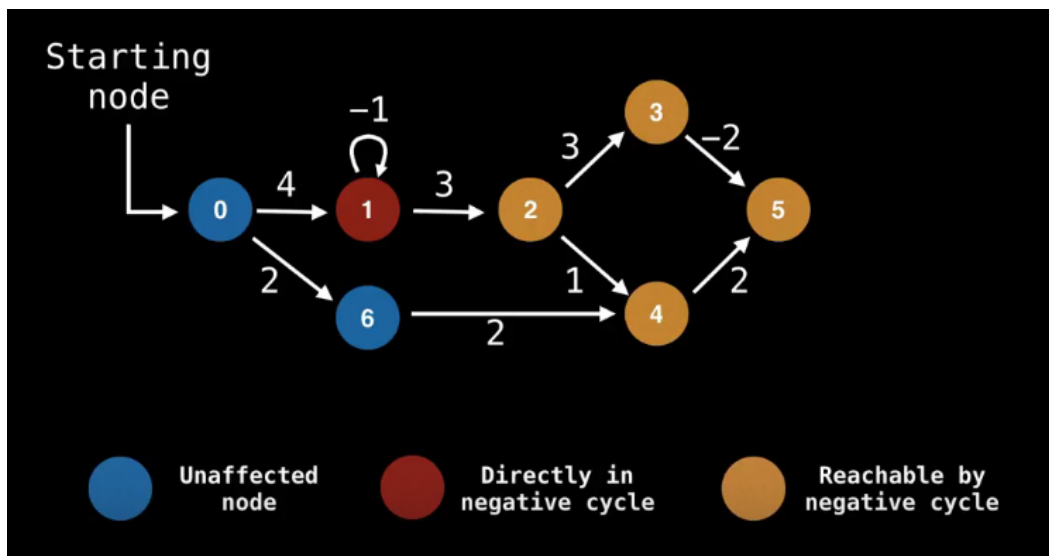
```

        g[i][j] = min(g[i][j], max(g[i][k],g[k][j]));
while(t--){
    int a, b;
    cin >> a >> b;
    if(g[a][b] != 0x3f3f3f3f)
        cout << g[a][b] << endl;
    else
        cout << -1 << endl;
}
return 0;
}

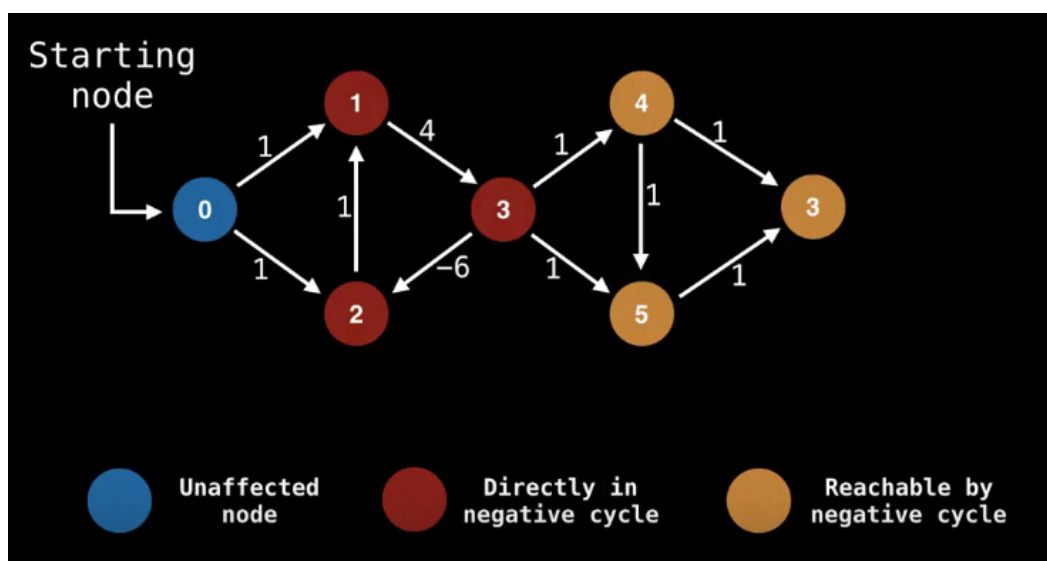
```

负环

负权边：权值为负数的边，称为负权边。



负环：环路中所有边的权值之和为负数，则称该环路为负环。



注意：带负环的图无法求最短路，因为可以沿着负环不停的循环，最短距离为负无穷大。

Bellman-Ford算法步骤

Bellman-Ford 算法采用动态规划（Dynamic Programming）进行设计，实现的时间复杂度为 $O(V * E)$ ，其中 V 为顶点数量， E 为边的数量。Dijkstra 算法采用贪心算法（Greedy Algorithm）范式进行设计，普通实现的时间复杂度为 $O(V^2)$ ，若基于堆优化的最小优先队列实现版本则时间复杂度为 $O(E + V \log V)$ 。

Bellman-Ford 算法描述：

1. 创建源顶点 v 到图中所有顶点的距离的集合 $dis[]$ ，为图中的所有顶点指定一个距离值，初始均为 ∞ ，源顶点距离为 0；
2. 计算最短路径，执行 $V - 1$ 次遍历（松弛边）；
对于图中的每条边：如果起点 u 的距离 d 加上边的权值 w 小于终点 v 的距离 d ，则更新终点 v 的距离值 d ；
3. 检测图中是否有负权边形成了环，遍历图中的所有边，如果 $dis[e.u] + e.w < dis[e.v]$ ，则说明存在环；

- 为什么要循环 $n - 1$ 次？

因为最短路径肯定是个简单路径，不可能包含回路的。而图有 n 个点，又不能有回路 所以最短路径最多 $n - 1$ 边，又因为每次循环，至少松弛了一条边 所以最多 $n - 1$ 次就行了。

- 为什么Dijkstra无法处理负边，而Bellman-Ford可以处理负边？

Dijkstra本质上是一种贪心策略，当有负边存在时，局部最优无法带来全局最优，贪心失效。

Bellman-Ford本质上是一种枚举策略，在求解 $s \rightarrow 0$ 的最短路径时，会计算所有 $s \rightarrow b$ 的不包含环路的路径，从中挑出权值和最小的路径。

SPFA 的核心原理和 Bellman-ford 算法是一样的，也是对点的松弛。只不过它优化了复杂度，优化的原理很简单，**只有被松弛过的点才有可能去松弛其他的点**。优化的方法也很简单，**用一个队列维护了可能存在新的松弛的点**，这样我们每次从这些点出发去寻找其他可以松弛的点加入队列。

SPFA 的代码也很短，实现起来难度很低，单单从代码上来看和普通的宽搜区别并不大。

SPFA算法步骤

1. 建立一个表格记录起始点到所有点的最短路径（该表格的初始值要赋为极大值，该点到他本身的路径赋为0）。
2. 建立一个队列，初始时队列里只有起始点。
3. 执行松弛操作，用队列里有的点作为起始点去刷新到所有点的最短路，如果刷新成功且被刷新点不在队列中则把该点加入到队列最后。重复执行直到队列为空。

值得注意的是

1. Bellman_ford 算法里最后 return-1 的判断条件写的是 $dist[n] > 0x3f3f3f3f/2$; 而 spfa 算法写的是 $dist[n] == 0x3f3f3f3f$; 其原因在于 Bellman_ford 算法会遍历所有的边，因此不管是不是和源点连通的边它都会得到更新；但是 SPFA 算法不一样，它相当于采用了 BFS，因此遍历到的结点都是与源点连通的，因此如果你要求的 n 和源点不连通，它不会得到更新，还是保持的 $0x3f3f3f3f$ 。
2. 由于 SPFA 算法是由 Bellman-ford 算法优化而来，在最坏的情况下时间复杂度和它一样即时间复杂度为 $O(nm)$ ，假如题目时间复杂度允许可以直接用 SPFA 算法去解 Dijkstra 算法的题目。
3. Bellman_ford 算法可以存在负权回路，是因为其循环的次数是有限制的因此最终不会发生死循环；但是 SPFA 算法不可以，由于用了队列来存储，只要发生了更新就会不断的入队，因此假如有负权回路请你不要用 SPFA 否则会死循环。

4. 求负环一般使用 SPFA 算法，方法是用一个 *cnt* 数组记录每个点到源点的边数，一个点被更新一次就 +1，一旦有点的边数达到了 *n* 那就证明存在了负环。

spfa判断负环

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int, int> PII;

const int N = 100010;

int n, m;
int dis[N], cnt[N]; // cnt: 统计最短路径经过的边数
bool st[N];
vector<PII> g[N];

int spfa(){
    queue<int> q;
    // 题中判断的是是否可能有负环，不是一定从 1 号点出发，所以将所有可能的点都加入
    // 注意:第一次出队的，自己到自己也可能存在负环
    for(int i = 1; i <= n; i++){
        st[i] = true;
        q.push(i);
    }

    while(q.size()){
        int t = q.front();
        q.pop();

        st[t] = false; // 取出的元素不在队列
        for(int i = 0; i < g[t].size(); i++){
            int j = g[t][i].second, w = g[t][i].first;
            if(dis[j] > dis[t] + w){
                dis[j] = dis[t] + w; // 更新距离
                cnt[j] = cnt[t] + 1; // 等于之前t的距离加上t ->
j的距离
                // 根据抽屉原理，说明经过某个节点两次，则说明有环
            }
        }
    }
}
```

```

        if(cnt[j] >= n) return true;
        if(!st[j]){ // 如果不在队列中
            q.push(j);
            st[j] = true;
        }
    }
}
return false;
}

int main(){
    ios::sync_with_stdio(0);
    cin >> n >> m;
    while(m --){
        int a, b, c;
        cin >> a >> b >> c;
        g[a].push_back(make_pair(c, b));
    }
    if(spfa()) puts("Yes");
    else puts("No");

    return 0;
}

```

蓝桥杯真题

环境治理（蓝桥杯C/C++2022A组国赛）

本题要求任意两点之间的最短路 $dis(i, j)$ ，那么我们可以看点数有多少个，可以看到 $1 \leq n \leq 100$ ，那么我们可以直接用Floyd算法在 $O(n^3)$ 时间复杂度上求解。

然后就是需要计算需要多少天了。实际上我们可以使用二分。

要想使用二分，肯定要先说明单调性，那么本题的天数是不是具有单调性呢？

显然是的，因为当前的 P 是随着天数是一个不增的变化，所谓不增的变化也就是 $P_{i+1} \leq P_i$ ，那么就能说明如果第 x 天 $P_x \leq Q$ ，那么对于后续的 $P_{x \rightarrow \infty} \leq Q$ 。

那么是不是我们直接对天数二分就可以了，然后怎么 check 呢？

实际上是不是每次 check 做一个 $O(n^3)$ 的遍历就可以了。我们先计算出来这个城市在经过 x 天后道路改善几次，然后就枚举各个城市，再求与这个城市相连的边做修改的变化，然后再跑Floyd最短路，再求我们当前的 P_x 即可。

最终复杂度是 $O(n^3 \log X^*)$ 其中 X^* 代表的是天数所二分的区间的长度。二分的左右边界可以设置为0和 10^9 ，因为 $0 \leq L_{i,j} \leq D_{i,j} \leq 10^5$ 所以每个城市道路改善最多能够有所变化的次数也就是 10^5

```
#include <stdio.h>
#include <string.h>
int d[105][105];
int limit[105][105];
int n;
int min(int a,int b)
{
    return a<b?a:b;
}
int calc(int day)           // 计算经过day 天治理后的P指标值
{
    int tmp[105][105];      // 用于治理的辅助数组
    int i,j,k;
    for (i =0; i<n; i++)
        for (j =0; j<n; j++)
            tmp[i][j]=d[i][j];
    for (i =0; i<n; i++)
    {
        int val = day/n+(day%n>=i+1?1:0);
        for (j =0 ; j<n; j++)
        {
            tmp[i][j]-=val;
            if (tmp[i][j]<limit[i][j]) tmp[i][j]=limit[i][j];
            tmp[j][i]-=val;
            if (tmp[j][i]<limit[j][i]) tmp[j][i]=limit[j][i];
        }
    }
}
```



```

    }
    for (k =0 ; k<n; k++)          // floyed 算法求最短路径
        for (i =0 ; i<n; i++)
            for (j =0 ; j<n; j++)
                tmp[i][j]=min(tmp[i][j],tmp[i][k]+tmp[k][j]);
    int res =0 ;
    for (i =0; i<n; i++)          // 计算治理后的P指标值
        for (j =0; j<n; j++)
            res+=tmp[i][j];
    return res ;
}

int main()
{
    int q;
    scanf("%d %d",&n,&q);
    int i,j;
    for (i =0; i<n; i++)
        for (j =0 ; j<n; j++)
            scanf("%d",&d[i][j]);
    for (i =0; i<n; i++)
        for (j =0 ; j<n; j++)
            scanf("%d",&limit[i][j]);
    int left =0,right= 100000*n,ans =-1;
    while (left<=right)
    {
        int mid = (left+right)/2;
        if (calc(mid)<=q)
        {
            right= mid -1;
            ans = mid;
        }
        else
            left=mid+1;
    }
    printf("%d\n",ans);
    return 0;
}

```

出差（蓝桥杯C/C++2022B组国赛）

把点权转换为边权即可。每个点需要额外的花费可以把它转换为入边上的边权。

剩下的只需要跑个Dijkstra最短路算法即可。

时间复杂度为 $O(M\log N)$

```
#include<bits/stdc++.h>
using namespace std;
#define pii pair<int,int>
const int maxn=1e5+5,inf=0x3f3f3f3f;
int n,m,add[maxn],dis[maxn];
bool vis[maxn]={0};
vector<pii> G[maxn];
void dijkstra()//Dijkstra
{
    for(int i=1;i<=n;i++) dis[i]=inf,vis[i]=0;
    dis[1]=0;
    priority_queue<pii,vector<pii>,greater<pii> >p;//堆优化
    p.push({0,1});
    while(!p.empty())
    {
        int u=p.top().second;
        p.pop();
        if(vis[u]) continue;
        vis[u]=1;
        for(int i=0;i<G[u].size();i++)
        {
            int cost=G[u][i].second,v=G[u][i].first;
            if(dis[v]>dis[u]+cost)
            {
                dis[v]=dis[u]+cost;
                p.push({dis[v],v});
            }
        }
    }
}
int main()
```

```

{
    cin>>n>>m;
    for(int i=1;i<=n;i++) cin>>add[i];
    for(int i=1;i<=m;i++)
    {
        int a,b,len;
        cin>>a>>b>>len;
        G[a].push_back({b,len+add[b]});
        G[b].push_back({a,len+add[a]});
        //vector存图，加上目的地点权
    }
    dijkstra();
    cout<<dis[n]-add[n]; //终点不用隔离
}

```

习题

选择最佳路线

解法一、反向建边

思路：从终点出发反向做一次最短路，然后在所有能选的起点中选最小值（既然朋友家的车站都叫 s 了，是不是在提示我们什么

注意一下反向加边。

spfa 和 dijkstra 都没问题，这里写的 dijkstra

时间复杂度： $O(Tm \log n)$

C++ 代码：

```

#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> pii;
const int M=2e5+5,N=1005;
int h[M],nex[M],to[M],w[M],cnt=1,n,m,aff,p,q,t,s,e,dist[N],ans;
bool vis[N];
inline void add(int a,int b,int c){

```

```

        to[cnt]=b,w[cnt]=c,nex[cnt]=h[a],h[a]=cnt++;
    }
    inline void init(){
        memset(vis,0,sizeof vis);
        memset(h,0,sizeof h);
        cnt=1;
        memset(dist,0x3f,sizeof dist);
        ans=0x3f3f3f3f;
    }

    void dijkstra(){
        priority_queue <pii,vector<pii>,greater<pii>>q;
        q.push({0,s});
        while(!q.empty()){
            auto hh=q.top();
            int nn=hh.second,dd=hh.first;
            q.pop();

            if(vis[nn])continue;
            vis[nn]=true;
            for(int i=h[nn];i;i=nex[i]){
                int ni=to[i];
                if(dist[ni]>dd+w[i]){
                    dist[ni]=dd+w[i];
                    q.push({dist[ni],ni});
                }
            }
        }
    }
}

int main(){
    while(cin>>n>>m>>s){
        init();
        for(int i=0;i<m;i++){
            scanf("%d%d%d",&p,&q,&t);
            add(q,p,t);
        }
        dist[s]=0;
        dijkstra();
        cin>>aff;
    }
}

```

```

        while(aff--){
            scanf("%d",&e);
            ans=min(ans,dist[e]);
        }
        printf("%d\n",ans==0x3f3f3f3f?-1:ans);
    }
    return 0;
}

```

解法二、超级源点

多个起点，不需要做多遍最短路，只需要构造一个超级源点，使其到其他起点的花费都是 0，以这点为起点做一遍最短路即可，图论中一种很常见的小技巧，

或者不加边，如果用 spfa 做的话，可以直接将所有起点放到队列中。

如果加边了，注意 N 和 M 的范围要多开一些。

```

#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> pii;
const int M=2e5+5,N=1005;
int h[M],nex[M],to[M],w[M],cnt=1,n,m,aff,p,q,t,s,e,dist[N],ans;
bool vis[N];
inline void add(int a,int b,int c){
    to[cnt]=b,w[cnt]=c,nex[cnt]=h[a],h[a]=cnt++;
}
inline void init(){
    memset(vis,0,sizeof vis);
    memset(h,0,sizeof h);
    cnt=1;
    memset(dist,0x3f,sizeof dist);
    ans=0x3f3f3f3f;
}

void dijkstra(){
    priority_queue <pii,vector<pii>,greater<pii>>q;
    q.push({0,s});
    while(!q.empty()){

```

```

        auto hh=q.top();
        int nn=hh.second,dd=hh.first;
        q.pop();
        //printf("%d %d\n",nn,dd);
        if(vis[nn])continue;
        vis[nn]=true;
        for(int i=h[nn];i;i=nex[i]){
            int ni=to[i];
            if(dist[ni]>dd+w[i]){
                dist[ni]=dd+w[i];
                q.push({dist[ni],ni});
            }
        }
    }
}

int main(){
    while(cin>>n>>m>>s){
        init();
        for(int i=0;i<m;i++){
            scanf("%d%d%d",&p,&q,&t);
            add(q,p,t);
        }
        dist[s]=0;
        dijkstra();
        cin>>aff;
        while(aff--){
            scanf("%d",&e);
            ans=min(ans,dist[e]);
        }
        printf("%d\n",ans==0x3f3f3f3f?-1:ans);
    }
    return 0;
}

```

最短路径

题目中 M 最大值为 500, 所以边的长度会超数据范围, 这里用快速幂处理下。

这题的关键点在于 **第 k 条路径的长度为 2^k** , 则有**第 k 条路径的长度会大于前 $k - 1$ 条路径的总和**, 所以:

- 在添加第 k 条路径时, 如果路径 k 连接的两个城市 A 、 B 已经通过其他路径间接的连通了, 那么第 k 条路径绝对不是 AB 间的最短路径
- 在添加第 k 条路径时, 如果路径 k 连接的两个城市 A 、 B 是第一次被连通了, 那么路径 k 就是 AB 之间的最短距离了
- 如何判断两个城市是否已经用其他路径间接连通了呢? 当然是并查集了。

问题: 取模运算会不会对求最短路造成影响呢?

回答: 不会。因为任何两个原本已经连通了的节点之间直接的边不会再建立, 因此最终构成的图是一棵树 (其实是原本图的 **最小生成树**), 因此任意两点间的路径唯一, 最短路径也唯一。

```
#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;

using LL = long long;
const int N = 110, M = 510, P = 100000;
int n, m;
int h[N], e[2*M], w[2*M], ne[2*M], idx;
int dist[N], p[N];
bool vis[N];

int qmi(int a, int b, int p){
    int res = 1;
    while(b){
        if(b & 1) res = (LL)res * a % p;
        b >>= 1;
        a = (LL)a * a % p;
    }
}
```

```

        return res % p;
    }

    int find(int x){
        if(x != p[x]) p[x] = find(p[x]);
        return p[x];
    }

    void add(int a, int b, int c){
        e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx++;
    }

    void dijkstra(){
        memset(dist, 0x3f, sizeof(dist));
        dist[0] = 0;

        for(int i = 1; i <= n; ++i){
            int t = -1;
            for(int j = 0; j < n; ++j){
                if(!vis[j] && (t == -1 || dist[t] > dist[j])){
                    t = j;
                }
            }
            vis[t] = true;
            for(int i = h[t]; i != -1; i = ne[i]){
                int j = e[i];
                if(dist[j] > dist[t] + w[i]){
                    dist[j] = dist[t] + w[i];
                }
            }
        }
    }

    int main()
    {
        cin >> n >> m;
        memset(h, -1, sizeof(h));

        for(int i = 0; i < n; ++i) p[i] = i;
    }

```



```

for(int i = 0; i < m; ++i){
    int a, b, c;
    cin >> a >> b;
    int aa = find(a), bb = find(b);
    if(aa != bb){
        p[aa] = bb;
        c = qmi(2, i, P);
        add(a, b, c);
        add(b, a, c);
    }
    else continue;
}

dijkstra();

for(int i = 1; i < n; ++i){
    if(dist[i] == 0x3f3f3f3f) cout << "-1" << endl;
    else cout << dist[i] % P << endl;
}

return 0;
}

```